

TaskTide: Backend-agnostic workflow orchestration system

Brendan J. Kenna ^{1¶}

¹ University College Dublin, Ireland ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 09 October 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

TaskTide is a backend-agnostic workflow orchestration system designed to mediate the deployment of user-defined data applications across different computing environments (Figure 1). In data-intensive environments, workflows are defined by users independently from the platform on which they are executed. This separation of concerns creates a natural role for orchestration middleware responsible for coordinating operational elements such as persistent state, execution context, and task lifecycle management.

Through its persistent workflow model, TaskTide enables users to register, query, and modify workflow deployments as they are consumed across distributed compute resources. These capabilities provide end users with operational controls that support real-time interaction with workflow state and execution context. As a result, workflows can be deployed across heterogeneous computing environments, including HPC, cloud, and containerized environments, while retaining a unified perspective on workflow state and the execution context of data-application deployments (Figure 4).

TaskTide treats workflow management, execution-context tracking, runtime introspection, and lifecycle influence as first-class orchestration concerns, while resource scheduling, provisioning, and workload execution remain infrastructural responsibilities. Workflow semantics remain user-defined, either at runtime or within the application itself. This separation allows TaskTide to operate as lightweight orchestration middleware that integrates with diverse computing environments and application domains, as demonstrated through use cases spanning bioinformatics, distributed application execution, and containerized AI workloads (Table 3).

Statement of Need

The growth of data-intensive research, services, and AI-driven applications has created new research opportunities for exploring how application execution can be coordinated across heterogeneous computing environments (Hop et al., 2024, 2026; Kenna et al., 2016; National Heart, Lung, and Blood Institute, 2016; Nicolas et al., 2018). Technologies such as Hadoop, SLURM, and Kubernetes provide essential infrastructure-level resource management and job-execution capabilities for modern computing platforms (Burns et al., 2016; Giri & Sharma, 2022; Salloum et al., 2016; Yoo et al., 2003). End users must still deploy and coordinate domain-specific *Data Application Layer Systems* (DALs) across these environments. In this work, the architectural separation between end-user DALs and service-provider *Infrastructure Layer Systems* (ILS) is referred to as the *Orchestration Layer System* (OLS), where workflow intent must be translated into distributed execution (Ramakrishnan et al., 2011).

As workflow orchestration is not inherently owned by either DALs or ILS (Figure 1), the relationship between application instances and their execution context is often implemented through infrastructure-specific scripts and utilities tightly coupled to a given ILS. This places

the burden of developing workflow state management, execution-context mapping, and lifecycle coordination on end users, reducing portability across computing environments. TaskTide addresses these operational concerns through a dedicated orchestration layer that persists workflow state independently of the underlying infrastructure (Figure 2). Beyond reducing workflow portability, the OLS gap introduces operational overhead for both end users and service providers. For end users, this creates development overhead for each workflow they need to deploy. For service providers, the OLS gap can blur the boundary between infrastructure-support responsibilities and workflow issues. These problems are amplified by the rapid evolution of application and infrastructure technologies, where tightly coupled solutions inherit platform-specific obsolescence risks.

These challenges highlight the need for flexible workflow orchestration systems designed to mediate the end-user experience without being tied to a specific backend technology. Such systems should decouple task scheduling from task execution and the underlying ILS, while elevating workflow state and execution context as first-class orchestration concerns that support real-time workflow introspection and lifecycle influence. TaskTide is a lightweight, daemon-less distributed workflow orchestration system for horizontally scaling DALS workloads across heterogeneous ILS. It operates across relational databases, NoSQL databases, and embedded databases. By treating workflow state as a persistent orchestration concern, TaskTide supports real-time workflow introspection and infrastructure-independent recovery. The same persistent state model enables decentralized worker coordination through optimistic task claiming with post-write ownership validation, removing the need for dedicated scheduling infrastructure.

State of the Field

The separate concerns of DALS and ILS have motivated a rich ecosystem of workflow engines, middleware systems, and pilot-job abstractions, each focusing on different aspects of the OLS boundary. Workflow systems such as Snakemake, Airflow, and Nextflow provide mature frameworks for expressing, scheduling, and executing workload sequences through workflow-specific abstractions (Koster & Rahmann, 2012; Langer et al., 2025; Yasmin et al., 2025). Complementing these systems, pilot-job frameworks such as DIRAC, WORCS, and PiCaS focus on decoupling workload execution from resource provisioning, enabling their use across diverse ILS (Boyer et al., 2022; SURF, 2025; Turilli et al., 2019).

With their differing focus, operational concerns such as task-lifecycle influence, execution-context mapping, and persistent workflow-state management are often addressed through combinations of workflow metadata, infrastructure adapters, execution environments, and application-specific utilities. Rather than representing these relationships through a dedicated operational abstraction, their implementation is typically influenced by the requirements of the surrounding workflow or infrastructure ecosystem.

TaskTide makes these operational concerns explicit through persistent workflow-state management by representing *Workflows*, *Steps*, *WorkItems*, *JobEnvironments*, and runtime metadata as first-class orchestration entities (Figure 3). This allows users to enrich TaskTide's data model with operational or domain-specific metadata while retaining real-time workflow introspection and lifecycle influence. With TaskTide's dynamic *Workflow* definition, execution behaviour, including parallelism and batch or service operation, and task acquisition, including step ordering and sequence flow, can be supplied at runtime. These operational controls provide end users with precise control over deployment behaviour without requiring changes to workflow definitions or underlying infrastructure mechanisms.

86 Software Design

87 Introduction

88 TaskTide is designed to bridge end-user DALs development and its scalable deployment across
89 heterogeneous ILS (Figure 1). It achieves this by decoupling task scheduling from execution
90 across ILS and maintaining associations between workflow intent and operational context
91 (Figures 2 and 3). The following sections describe how TaskTide's architecture and operational
92 model map to practical value for end users and service providers as an OLS solution (Table 1).

93 TaskTide Data Model

94 TaskTide defines a standardized data model that enables reproducible and traceable DALs
95 deployments. The model captures workflow intent and execution context through the persistent
96 representation of *Workflows*, *Steps*, *WorkItems*, and *JobEnvironments*, together with their
97 associated metadata (Figure 3). Motivated by the extraction, transformation, and loading
98 pattern, a *Workflow* is a collection of distinct but related operations to be performed, such
99 as an ETL sequence. Each distinct operation of a given *Workflow* is a *Step*, while individual
100 *WorkItems* represent independently executable units of that *Step* (Gropp et al., 1996; Singh,
101 2022; Venkateswarlu & Vasista, 2023). This structure expresses *Workflows* as referential
102 collections of independently executable *WorkItems*, making workflow distribution and horizontal
103 scaling explicit aspects of the orchestration model. With its persistence model, TaskTide
104 enables workflow state, execution progress, and execution contexts to be managed as first-class
105 orchestration concerns while providing a shared coordination substrate for distributed worker
106 execution.

107 To support workflow introspection and operational traceability, TaskTide persists task-lifecycle
108 information and execution-context metadata. These include active *WorkItem* state, execution
109 logs, scheduling identifiers, and host-specific parameters that collectively establish direct
110 linkages between an application and its execution environment. Such mappings enable *Workflow*
111 introspection, monitoring, runtime mutability, and reproducibility across heterogeneous ILS
112 while remaining extensible through operational or domain-specific metadata.

113 TaskTide Persistence and Execution Model

114 TaskTide coordinates workflow execution through a database-backed producer-consumer pattern
115 (Figure 2), in which *EngineWorker* instances poll runnable *WorkItems* from a centralized
116 repository rather than receiving explicit task assignments (Figure 4). Acquisition uses optimistic
117 task claiming with post-write ownership validation, allowing decentralized task discovery without
118 dedicated scheduling infrastructure. This complements TaskTide's backend-agnostic persistence
119 layer, which supports relational and non-relational database systems and persists workflow
120 state, task-lifecycle tracking, and execution metadata across backend technologies (Figure 3).

121 TaskTide's persisted workflow state enables incomplete *WorkItems* to be identified and
122 reassigned independently of worker failures (Figure 2). Horizontal scaling is supported by
123 deploying additional *EngineWorker* instances to consume available *WorkItems* from the shared
124 repository without requiring workflow redistribution (Table 2). This repository-driven execution
125 model remains independent of infrastructure schedulers and provisioning mechanisms (Figure
126 1).

127 User-defined execution policies determine how *EngineWorker* instances traverse and poll
128 *WorkItems*, allowing workflows to be processed sequentially, cyclically, or by targeting a *Step*
129 without redefining the *Workflow*. Integrating execution policies with *WorkItem* lifecycle states
130 enables real-time, dependency-aware influence over *Workflow* progression. Lifecycle transitions
131 are coordinated through the *EngineObserverChain*, which validates task progress and relays
132 updates to the repository for real-time monitoring (Figure 4).

133 Deployment

134 TaskTide supports deployment across heterogeneous ILS environments, including HPC, cloud,
135 containerized, and distributed-service configurations. Its modular architecture separates
136 workflow orchestration from execution infrastructure, allowing the *Manager* and *Engine*
137 components to be deployed independently according to the requirements of a service
138 environment (Table 1). A unified client provides a common mechanism for configuring these
139 components, while backend-agnostic dependencies enable operation across diverse database
140 and infrastructure environments (Table 2).

141 Deployment-specific configuration is externalized through command-line options and
142 conventional application-properties files. Execution policies, workflow sequences, and batch
143 or service operations can therefore be adapted per instance while coordinating through a
144 centralized persistence layer (Figure 2). The same configuration model supports standalone
145 batch execution, service operation across server farms, or web-service deployment. By
146 separating workflow definition, orchestration, and infrastructure concerns, TaskTide reduces
147 platform-specific integration and the need for bespoke orchestration solutions while improving
148 workflow portability across heterogeneous ILS.

149 Relation to the Pilot-Job Abstraction

150 TaskTide's design was motivated by treating an ETL as the base unit of work for the pilot-job
151 abstraction (Figures 2 and 5). The combination provides a natural separation between what
152 is executed and how it is scaled: ETL workflows encapsulate data processing, while pilots
153 provide execution capacity. Within TaskTide, these patterns are represented as persistent
154 operational elements. ETL elements are captured through actionable data models, namely
155 *Workflows*, *Steps*, and *WorkItems*, while pilot-job execution fleets are represented through
156 *JobEnvironments* and associated metadata.

157 TaskTide persists actionable ETL elements within a centralized repository, from which
158 they are consumed by *TaskTide-Engine* instances deployed across pilot-job fleets. This
159 separation expresses workload scaling through a common operational model while resource
160 provisioning remains the responsibility of the underlying ILS (Figure 5). Scheduler-specific
161 provisioning mechanisms can be represented through TaskTide configurations and delegated to
162 the native scheduling interfaces of the lower ILS (Table 2). Workload dispatch is coordinated
163 by the *TaskTide-Repository*, while execution is performed by *TaskTide-Engine* instances
164 deployed within provisioned ILS jobs (Figure 2). This separates workflow-state management
165 and execution-context tracking from resource scheduling and provisioning, which remain
166 responsibilities of the lower ILS (Figures 1 and 5).

167 In TaskTide, *EngineWorkers* select tasks rather than receiving explicit assignments. Workers
168 independently attempt ownership of runnable tasks. After persisting a unique lock identifier, a
169 worker re-polls the task and proceeds only when the persisted ownership record matches its
170 claim. This decentralized protocol allows concurrent task discovery, avoids dedicated scheduling
171 infrastructure, and ensures that only successfully validated owners execute claimed *WorkItems*.
172 TaskTide does not claim global exactly-once processing semantics; it instead relies on the
173 ownership protocol and workflow design to maintain execution correctness.

174 The ETL and pilot-job patterns provide complementary forms of scaling. At the ETL level, the
175 number of tasks within a *Step* can reflect practical data-slicing requirements, while workflow
176 definitions retain related data-wrangling and cleaning tasks as explicit operational stages.
177 At the pilot-job level, execution capacity can be adjusted according to the number of tasks
178 processed by each instance and the desired degree of serial or parallel execution. Together,
179 these scaling dimensions provide a common operational model between DALS applications and
180 their deployment across service-provider ILS environments. By separating workflow definition,
181 pilot provisioning, and workflow processing, TaskTide remains backend-agnostic while retaining
182 the scalability, flexibility, real-time introspection, and workflow-level influence associated with

183 the combined ETL and pilot-job model.

184 **Research Impact Statement**

185 TaskTide originated from workflow-orchestration requirements encountered during data-
186 engineering work supporting large-scale biomedical research projects (Hop et al., 2024, 2026;
187 Kenna et al., 2016; Nicolas et al., 2018). The author contributed to these projects as a data
188 engineer, where recurring requirements included workflow-state management, execution-context
189 tracking, lifecycle coordination, and deployment across heterogeneous computing resources.
190 These projects are cited as provenance for the system's requirements rather than as evidence
191 of TaskTide usage.

192 TaskTide is distributed with source code, packages, container images, deployment
193 documentation, API references, and representative use cases spanning bioinformatics,
194 distributed execution, and containerized AI workloads (Table 3), supporting independent
195 evaluation, reuse, and adoption.

196 **AI Usage Disclosure**

197 The design, architecture, implementation, evaluation, and scientific claims of TaskTide were
198 developed by the author. AI tools were not used to design the TaskTide workflow-orchestration
199 model, scheduling approach, state-management mechanism, or system architecture. ChatGPT
200 and Microsoft Copilot were used during manuscript preparation to improve readability and
201 presentation. AI-generated outputs were reviewed and adapted by the author before inclusion.
202 Auxiliary demonstration code was generated for selected non-core example workloads.

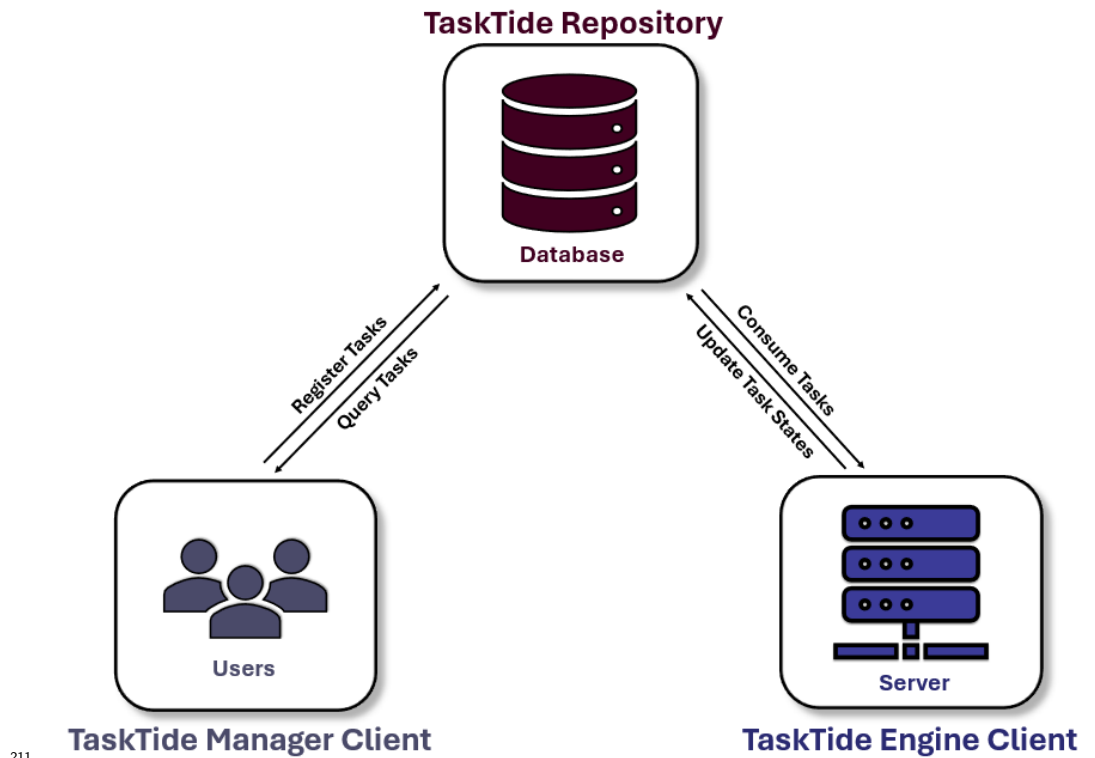
203 **Figures and Tables**

204 **Figure 1: High-Level Layered Architecture**



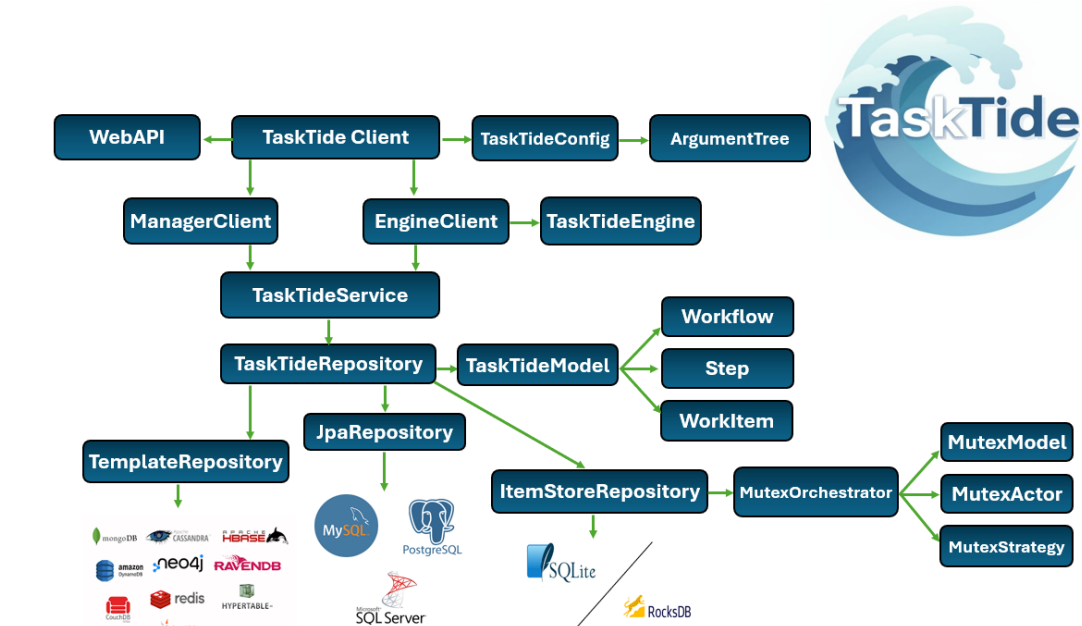
205
206 **Figure 1:** Shows the positioning of the **TaskTide Orchestration Layer System**. This middleware
207 sits between customer **Data Application Layer** development and **Infrastructure Layer** execution,
208 mediating the sessionless scale-out of DALS instances across heterogeneous infrastructure
209 layers, as demonstrated by selected use cases at tasktide.org/use-cases.

210 **Figure 2: TaskTide Operating Principle**



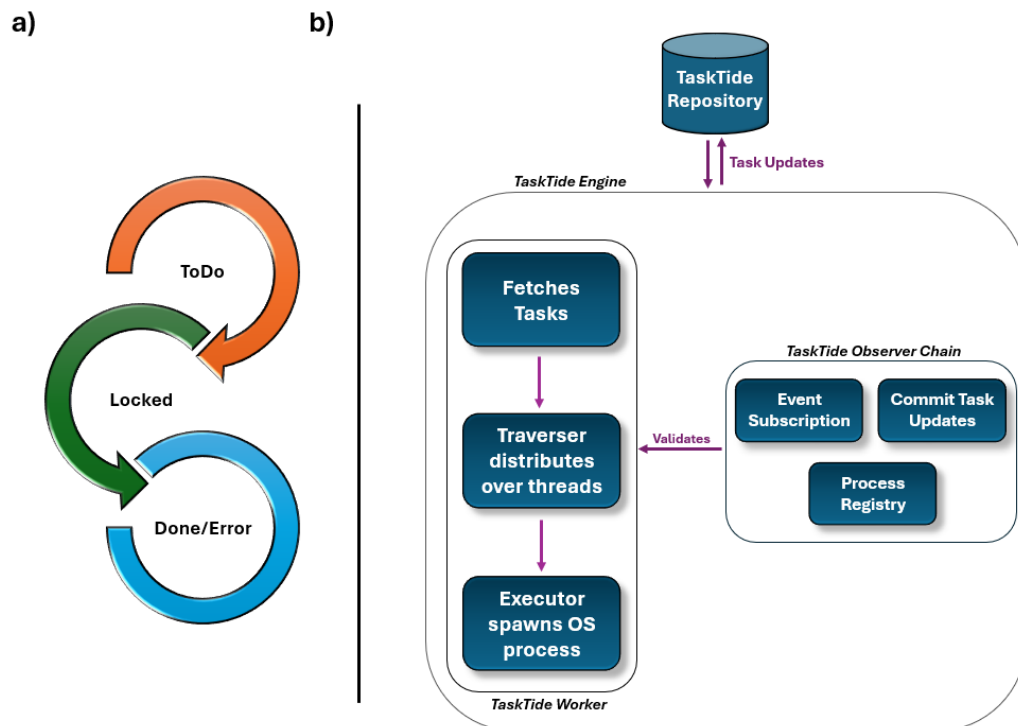
211 **Figure 2:** Shows the **producer-consumer** operating principle of the TaskTide workflow engine.
 212 **1) The TaskTide Manager** command-line client is the interface through which users register
 213 and query tasks across user-defined workflows. **2) The TaskTide Repository** is the ambassador
 214 interface that manages task storage and retrieval and persists its data to the configured backend
 215 database, whether NoSQL or SQL. **3) The TaskTide Engine** command-line client consumes
 216 the required task set or sets of tasks.
 217

218 **Figure 3: TaskTide Architecture**



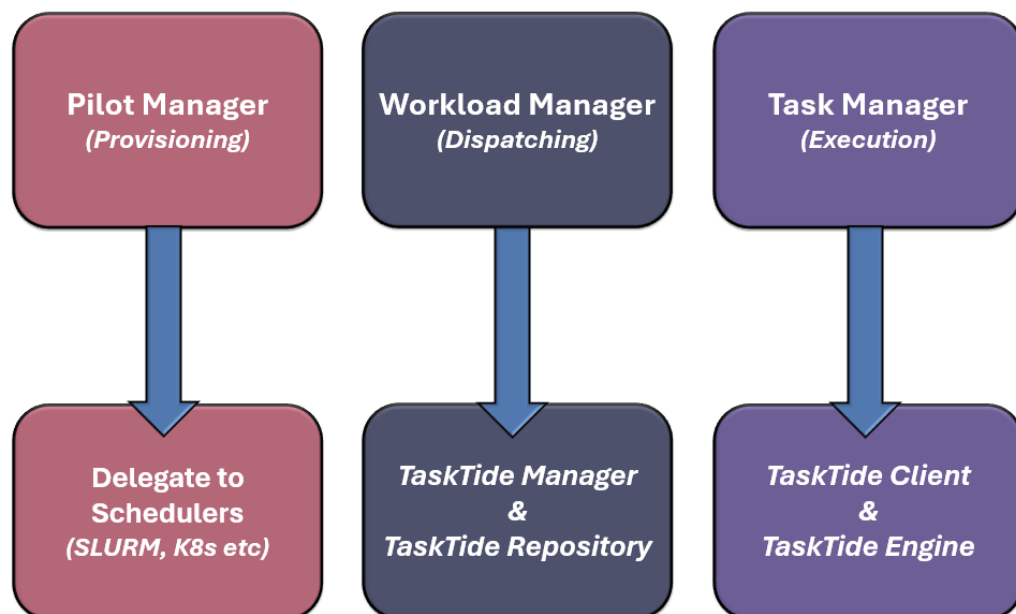
219
220 **Figure 3: Shows the architecture of the TaskTide Workflow Orchestration Engine.**

221 **Figure 4: TaskTide WorkItem Lifecycle**



222
223 **Figure 4:** Shows the persistent lifecycle of tasks registered in TaskTide (6-a), which is enacted
224 by the TaskTide Engine (6-b). **1) The TaskTide Engine** fetches the configured workload
225 from the TaskTide Repository. **2) The TaskTide Engine Worker** distributes available tasks
226 across configured thread pools for parallel processing. **3) TaskTide Engine Executor** instances
227 spawn operating-system processes for task execution and expose data-model identifiers as
228 environmental variables for tasks to use. **4) The TaskTide Engine Observer Chain** subscribes
229 to key lifecycle transition points, namely onTaskStart, onTaskProcessing, and onTaskEnd,
230 and performs related task validation (6-a).

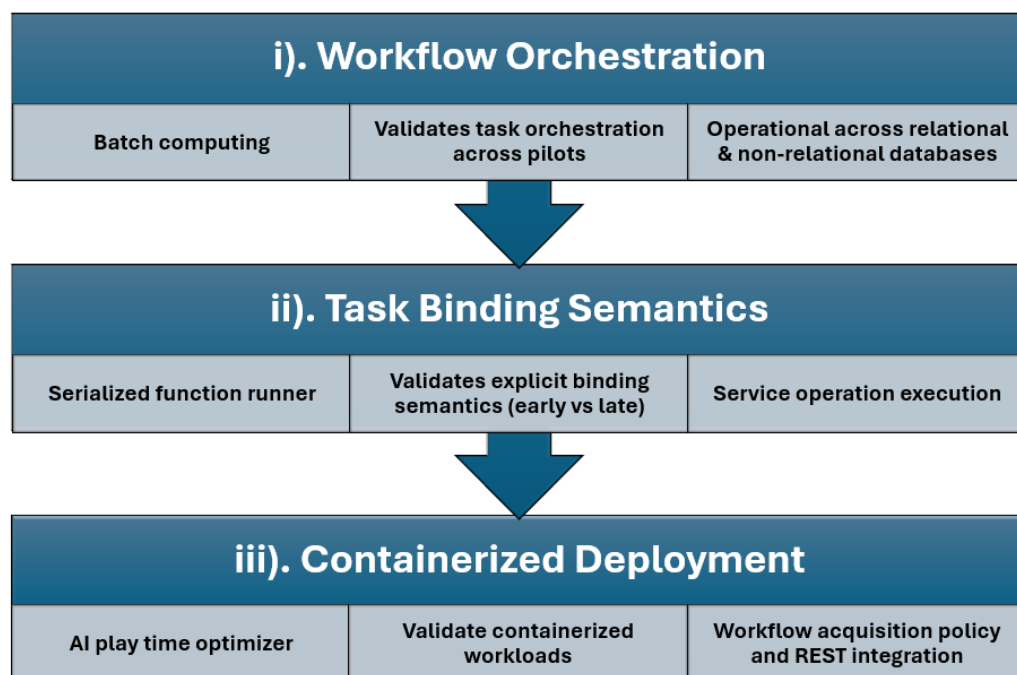
231 **Figure 5: Mapping TaskTide to the Pilot-Job Abstraction**



232

233 **Figure 5:** Shows how TaskTide's sessionless internals map to the pilot-job abstraction. **1)**
 234 **Pilot provisioning** is delegated to the underlying platform, as task tracking is more relevant to
 235 TaskTide as DALS intent than a given job as an ILS concern. **2) Workload management** is
 236 implemented by the TaskTide Repository, where the configured database acts as the single
 237 source of truth for **task lifecycle management** and where **binding semantics** are expressed. **3)**
 238 **Task execution** is performed by instances or fleets of the TaskTide Engine client. The engine
 239 **polls available tasks** from configured or arbitrary collections, executes their units of work, and
 240 feeds back updates observed across the execution process in real time, including logs and
 241 current and completed state.

242 **Figure 6: High-Level Development Approach**



243
244 **Figure 6:** Shows the **iterative and incremental** development approach used for TaskTide. Each
245 stage had key deliverables supported by a use-case implementation. **1) Workflow orchestration**
246 **was verified using bioinformatics pipelines. 2) Task binding semantics** were explicitly verified
247 **using a serialized Julia function runner for early task binding and a SparkR image-analysis**
248 **application for late task binding. 3) Containerized deployment** was verified using an AI
249 **play-time optimizer Python application.**

250 **Table 1: TaskTide Feature Summary**

TaskTide Feature Summary			
Component	Purpose	Contributions	Operational Value
Core Library	Defines data model, and CRUD operations	Backend agnostic; Standardized data model; Custom data fields; Workflow-Execution context mapping	Workflow reproducibility & portability; Flexible workflow introspection; Heterogenous deployment
Engine Library	Defines modularized components for consuming registered tasks	Producer-consumer execution; Pilot-Job semantics; Multi-Level parallelism; Real-time monitoring.	Scalable ETL deployment; Late & early task binding semantics; Adaptive workflows; Fault tolerance.
Web API	Defines RESTful interface for interacting with TaskTide services	Unified access layer for workflow management; Supports programmatic task registration, querying, and monitoring; Enables integration with external systems & services	Enables deployment as a service within distributed architectures; Supports web-based dashboarding and multi-user access
Client Application	Exposes Manager, and Engine functionality through a single interface	Unified client; Configurable; Extensible	Low-friction adoption; Automation; Multi disciplinary application

251
252 **Table 1: Summarizes the feature contributions of TaskTide's modules and translates these**
253 **features into their operational value.**

DRAFT

254 Table 2: TaskTide Configurations

Task Tide Configurations				
Client	Property	Operand	Example Value	Description
Global	Client	tasktide client	manager	TaskTide client to use
	Repository Type	tasktide core repository type	sqlite	Backend DB to use
Manager	Target	tasktide manager target	WORKITEM	TaskTide model for CRUD request
	Method	tasktide manager method	Summarize	Manager CRUD operation to run
Engine	WorkItem Threads	tasktide engine workitem threads	2	Concurrent workers
	Execution Policy	tasktide engine execution-policy	BATCH SERVICE	How engine runs
Web-API	Port	tasktide web-api server port	8080	Port of web server
	Base Path	tasktide web-api server base-path	tasktide	Path of web server
Mutex	Lock Directory	tasktide mutex lockDir	/tmp/tasktide-mutex	Where mutex operates
	Stale File Threshold	tasktide staleFileThreshold	5000	Elapsed milliseconds flagging stagnancy

255
256 Table 2: Shows an example set of TaskTide Client Configuration properties.

257 **Table 3: TaskTide Resources**

TaskTide Resources	
Resource	Location
Project Website	tasktide.org
Use Cases	use-cases.tasktide.org
Source Code	github.tasktide.org
Docker Image	docker.tasktide.org
Maven Packages	maven.tasktide.org
Documentation	docs.tasktide.org
API Documentation	api-docs.tasktide.org

258

259 **Table 3:** Summarizes the primary **TaskTide** resources. High-level resources provide project
260 overviews and representative use cases for prospective users, while technical resources provide
261 source code, package dependencies, implementation guidance, deployment documentation,
262 and API references for software developers and service providers.

- 263 Boyer, A. F., Haen, C., Stagni, F., & Hill, D. R. C. (2022). DIRAC site director: Improving
264 pilot-job provisioning on grid resources. *Future Generation Computer Systems*, 133.
265 <https://doi.org/10.1016/j.future.2022.03.002>
- 266 Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, omega, and
267 kubernetes. *Communications of the ACM*, 59(5), 50–57. <https://doi.org/10.1145/2890784>
- 268 Giri, P. R., & Sharma, G. (2022). Apache hadoop architecture, applications, and hadoop
269 distributed file system. *Semiconductor Science and Information Devices*, 4(1). <https://doi.org/10.30564/ssid.v4i1.4619>
- 270
- 271 Gropp, W., Lusk, E., Doss, N., & Skjellum, A. (1996). A high-performance, portable
272 implementation of the MPI message passing interface standard. *Parallel Computing*, 22(6),
273 789–828. [https://doi.org/10.1016/0167-8191\(96\)00024-5](https://doi.org/10.1016/0167-8191(96)00024-5)
- 274 Hop, P. J., Kooyman, M., Kenna, B. J., Zwamborn, R. A. J., Eijk, K. R. van, Wang, Y., & others.
275 (2026). Large-scale exome analyses reveal new rare variant contributions in amyotrophic
276 lateral sclerosis. *Nature Genetics*, 58(4), 717–725. <https://doi.org/10.1038/s41588-026-02535-9>
- 277
- 278 Hop, P. J., Lai, D., Keagle, P. J., Baron, D. M., Kenna, B. J., Kooyman, M., & others.
279 (2024). Systematic rare variant analyses identify RAB32 as a susceptibility gene for familial
280 parkinson's disease. *Nature Genetics*, 56(7), 1371–1376. <https://doi.org/10.1038/s41588-024-01787-7>
- 281
- 282 Kenna, K. P., Van Doormaal, P. T. C., Dekker, A. M., Ticozzi, N., Kenna, B. J., Diekstra, F.
283 P., & others. (2016). NEK1 variants confer susceptibility to amyotrophic lateral sclerosis.
284 *Nature Genetics*, 48(9). <https://doi.org/10.1038/ng.3626>
- 285 Koster, J., & Rahmann, S. (2012). Snakemake: A scalable bioinformatics workflow engine.
286 *Bioinformatics*, 28(19). <https://doi.org/10.1093/bioinformatics/bts480>
- 287 Langer, B. E., Amaral, A., Baudement, M. O., Bonath, F., Charles, M., Chitneedi, P. K.,
288 & others. (2025). Empowering bioinformatics communities with Nextflow and nf-core.
289 *Genome Biology*, 26(1). <https://doi.org/10.1186/s13059-025-03673-9>
- 290 National Heart, Lung, and Blood Institute. (2016). *NHLBI trans-omics for precision medicine*
291 *whole genome sequencing program*. <https://www.nhlbiwgs.org/>
- 292 Nicolas, A., Kenna, K. P., Renton, A. E., Ticozzi, N., Faghri, F., Chia, R., & others.
293 (2018). Genome-wide analyses identify KIF5A as a novel ALS gene. *Neuron*. <https://doi.org/10.1016/j.neuron.2018.02.027>
- 294
- 295 Ramakrishnan, L., Chase, J. S., Gannon, D., Nurmi, D., & Wolski, R. (2011). Deadline-
296 sensitive workflow orchestration without explicit resource control. *Journal of Parallel and*
297 *Distributed Computing*, 71(3), 343–353. <https://doi.org/10.1016/j.jpdc.2010.11.010>
- 298 Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics
299 on apache spark. *International Journal of Data Science and Analytics*, 1(3–4). <https://doi.org/10.1007/s41060-016-0027-9>
- 300
- 301 Singh, M. M. (2022). Extraction transformation and loading (ETL) of data using ETL tools.
302 *International Journal for Research in Applied Science and Engineering Technology*, 10(6).
303 <https://doi.org/10.22214/ijraset.2022.44939>
- 304 SURF. (2025). *PiCaS: CouchDB-backed pilot job framework*. <https://pypi.org/project/picas/>
- 305 Turilli, M., Santcroos, M., & Jha, S. (2019). A comprehensive perspective on pilot-job systems.
306 *ACM Computing Surveys*, 51(2). <https://doi.org/10.1145/3177851>
- 307 Venkateswarlu, M., & Vasista, T. G. (2023). Extraction, transformation and loading process
308 in the cloud computing scenario. *International Journal of Engineering Applied Sciences*
309 *and Technology*, 8(1). <https://doi.org/10.33564/ijeast.2023.v08i01.036>

- 310 Yasmin, J., Wang, J. A., Tian, Y., & Adams, B. (2025). An empirical study of developers'
311 challenges in implementing workflows as code: A case study on apache airflow. *Journal of*
312 *Systems and Software*, 219. <https://doi.org/10.1016/j.jss.2024.112248>
- 313 Yoo, A. B., Jette, M. A., & Grondona, M. (2003). SLURM: Simple linux utility for resource
314 management. *Job Scheduling Strategies for Parallel Processing*, 2862. [https://doi.org/10.](https://doi.org/10.1007/10968987_3)
315 [1007/10968987_3](https://doi.org/10.1007/10968987_3)

DRAFT